

Menu-Driven Student Name Operations

using String Handling Functions in C

Submitted By

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16

1. Problem Statement

Write a menu-driven C program that reads a student's first name, middle name and last name, and performs a set of string operations. Every operation must be written as a separate user-defined function. Call by reference (pointers) must be used wherever a function modifies a string or returns a result to the caller.

2. Objectives

- To understand string handling in C using the standard library functions of `<string.h>` and `<ctype.h>`.
- To practise modular programming by writing a separate user-defined function for each task.
- To understand call by reference by passing pointers to functions that modify strings or return values.
- To build a menu-driven program using do-while and switch-case.

3. Header Files and Library Functions Used

Header File	Functions / Purpose
stdio.h	printf(), scanf(), getchar() – input and output
string.h	strcpy(), strcat(), strlen(), strcmp(), strtok(), strstr() – string handling
ctype.h	isalpha(), isdigit(), isspace(), tolower() – character classification and conversion

4. Menu Options

Option	Operation
1	Combine first, middle and last name and display the full name
2	Find and display the length of the full name
3	Copy the full name into another string and display it
4	Compare the first name and last name (equal or different)
5	Display the name in reverse order (last, middle, first)
6	Count vowels, consonants, digits and spaces in the full name
7	Search a word in the full name
8	Exit

5. User-Defined Functions

The table below lists every function used in the program with the library functions it relies on.

Function Prototype	Task	Library Functions
<code>void combineNames(const char firstName[], const char middleName[], const char lastName[], char fullName[]);</code>	1	<code>strcpy()</code> , <code>strcat()</code>
<code>void findLength(const char fullName[], size_t *length);</code>	2	<code>strlen()</code>
<code>void copyName(const char source[], char destination[]);</code>	3	<code>strcpy()</code>
<code>void compareNames(const char firstName[], const char lastName[], int *comparisonResult);</code>	4	<code>strcmp()</code>
<code>void reverseNameOrder(const char firstName[], const char middleName[], const char lastName[], char reverseName[]);</code>	5	<code>strcpy()</code> , <code>strcat()</code> , <code>strtok()</code>
<code>void countCharacters(const char fullName[], int *vowels, int *consonants, int *digits, int *spaces);</code>	6	<code>strlen()</code> , <code>isalpha()</code> , <code>isdigit()</code> , <code>isspace()</code> , <code>tolower()</code>
<code>void searchWord(const char fullName[], const char word[], int *searchResult);</code>	7	<code>strstr()</code> , <code>tolower()</code>

Helper functions: `displayMenu()` prints the menu and `clearInputBuffer()` removes leftover characters from the input buffer so that the next input is read correctly.

6. Call by Reference

In C, arguments are passed by value. To let a function change a variable of the caller, or return more than one result, the address of the variable is passed and the function works through a pointer. This program uses the idea in the following ways:

- **Strings (arrays):** an array name is already the address of its first element, so functions such as `combineNames()`, `copyName()` and `reverseNameOrder()` directly fill the destination array.
- **Single results:** `findLength()`, `compareNames()` and `searchWord()` receive the address of a variable (`&length`, `&comparisonResult`, `&searchResult`) and store their answer through the pointer.
- **Multiple results:** `countCharacters()` returns four values (vowels, consonants, digits, spaces) using four pointer parameters.

7. Working of the Program

7.1 Algorithm

1. Start.
2. Read the first name, middle name and last name.
3. Call `combineNames()` to build the full name.

4. Display the menu and read the user's choice.
5. Using switch–case, call the function for the selected option and display its result.
6. Repeat steps 4–5 until the user selects option 8 (Exit).
7. Stop.

7.2 Logic of Each Function

- **combineNames():** copies the first name with strcpy() and appends a space, the middle name, another space and the last name with strcat().
- **findLength():** stores strlen(fullName) in the variable pointed to by length.
- **copyName():** copies the source string into the destination using strcpy().
- **compareNames():** stores the result of strcmp() through the pointer. A result of 0 means the two names are equal; any other value means they are different.
- **reverseNameOrder():** builds “First Middle Last” with strcpy() and strcat(), splits it into words with strtok(), and joins the words from last to first with strcat().
- **countCharacters():** converts each character to lower case with tolower() and checks it. Letters a, e, i, o, u are vowels, other letters (isalpha) are consonants; isdigit() counts digits and isspace() counts spaces.
- **searchWord():** converts the name and the word to lower case (so the search ignores case) and uses strstr(). The pointer receives the index of the match, or –1 if the word is not present.

8. Source Code

File name: student_name_menu.c

```

1  /*
2  * Program : Student Name Operations (Menu Driven)
3  * Author  : Rudra Pratap Singh
4  * SAP ID  : 590032811
5  * Batch   : 16
6  *
7  * Description:
8  *   Reads a student's first, middle and last name and performs various
9  *   string operations using user-defined functions. Pointers (call by
10 *   reference) are used wherever a function modifies a string or returns
11 *   a result to the caller.
12 */
13
14 #include <stdio.h>
15 #include <string.h>
16 #include <ctype.h>
17
18 #define NAME_SIZE 50
19 #define FULL_SIZE 160
20
21 /* ----- Function prototypes ----- */
22 void combineNames(const char firstName[], const char middleName[],
23                  const char lastName[], char fullName[]);
24 void findLength(const char fullName[], size_t *length);
25 void copyName(const char source[], char destination[]);
26 void compareNames(const char firstName[], const char lastName[],
27                  int *comparisonResult);
28 void reverseNameOrder(const char firstName[], const char middleName[],
29                      const char lastName[], char reverseName[]);
30 void countCharacters(const char fullName[], int *vowels, int *consonants,
31                    int *digits, int *spaces);
32 void searchWord(const char fullName[], const char word[], int *searchResult);
33 void displayMenu(void);
34 void clearInputBuffer(void);
35
36 /* ----- main ----- */
37 int main(void)
38 {
39     char firstName[NAME_SIZE], middleName[NAME_SIZE], lastName[NAME_SIZE];
40     char fullName[FULL_SIZE], copiedName[FULL_SIZE];
41     char reverseName[FULL_SIZE], word[NAME_SIZE];
42     size_t length;
43     int choice, comparisonResult, searchResult;
44     int vowels, consonants, digits, spaces;
45
46     printf("=====\n");
47     printf("      STUDENT NAME OPERATIONS\n");
48     printf("=====\n");
49
50     printf("Enter first name  : ");
51     scanf("%49s", firstName);
52     printf("Enter middle name : ");
53     scanf("%49s", middleName);
54     printf("Enter last name   : ");
55     scanf("%49s", lastName);
56     clearInputBuffer();
57
58     /* Build the full name once so every menu option can use it */
59     combineNames(firstName, middleName, lastName, fullName);
60
61     do
62     {
63         displayMenu();
64         printf("Enter your choice (1-8): ");
65         if (scanf("%d", &choice) != 1)
66         {
67             clearInputBuffer();

```

```

68         choice = 0;                                /* force "invalid choice" */
69     }
70     clearInputBuffer();
71     printf("\n");
72
73     switch (choice)
74     {
75     case 1:
76         combineNames(firstName, middleName, lastName, fullName);
77         printf("Full name      : %s\n", fullName);
78         break;
79
80     case 2:
81         findLength(fullName, &length);
82         printf("Length of full name (with spaces) : %zu\n", length);
83         break;
84
85     case 3:
86         copyName(fullName, copiedName);
87         printf("Copied string   : %s\n", copiedName);
88         break;
89
90     case 4:
91         compareNames(firstName, lastName, &comparisonResult);
92         if (comparisonResult == 0)
93             printf("First name and last name are EQUAL.\n");
94         else
95             printf("First name and last name are DIFFERENT.\n");
96         break;
97
98     case 5:
99         reverseNameOrder(firstName, middleName, lastName, reverseName);
100        printf("Reverse order    : %s\n", reverseName);
101        break;
102
103    case 6:
104        countCharacters(fullName, &vowels, &consonants, &digits, &spaces);
105        printf("Vowels       : %d\n", vowels);
106        printf("Consonants  : %d\n", consonants);
107        printf("Digits      : %d\n", digits);
108        printf("Spaces     : %d\n", spaces);
109        break;
110
111    case 7:
112        printf("Enter the word to search: ");
113        scanf("%49s", word);
114        clearInputBuffer();
115        searchWord(fullName, word, &searchResult);
116        if (searchResult >= 0)
117            printf("The word \"%s\" FOUND in the full name (position %d).\n",
118                word, searchResult + 1);
119        else
120            printf("The word \"%s\" NOT FOUND in the full name.\n", word);
121        break;
122
123    case 8:
124        printf("Exiting the program. Thank you!\n");
125        break;
126
127    default:
128        printf("Invalid choice! Please enter a number between 1 and 8.\n");
129    }
130    } while (choice != 8);
131
132    return 0;
133 }
134
135 /* ----- Function definitions ----- */
136
137 /* Task 1: join first, middle and last name with single spaces */
138 void combineNames(const char firstName[], const char middleName[],
139                 const char lastName[], char fullName[])

```

```

140 {
141     strcpy(fullName, firstName);
142     strcat(fullName, " ");
143     strcat(fullName, middleName);
144     strcat(fullName, " ");
145     strcat(fullName, lastName);
146 }
147
148 /* Task 2: length of the full name; result returned through pointer */
149 void findLength(const char fullName[], size_t *length)
150 {
151     *length = strlen(fullName);
152 }
153
154 /* Task 3: copy source string into destination */
155 void copyName(const char source[], char destination[])
156 {
157     strcpy(destination, source);
158 }
159
160 /* Task 4: compare first and last name; strcmp() result via pointer */
161 void compareNames(const char firstName[], const char lastName[],
162                  int *comparisonResult)
163 {
164     *comparisonResult = strcmp(firstName, lastName);
165 }
166
167 /* Task 5: build "Last Middle First" using strcpy, strcat and strtok */
168 void reverseNameOrder(const char firstName[], const char middleName[],
169                      const char lastName[], char reverseName[])
170 {
171     char temp[FULL_SIZE];
172     char *words[3];
173     char *token;
174     int count = 0, i;
175
176     /* Build "First Middle Last" in a temporary string */
177     strcpy(temp, firstName);
178     strcat(temp, " ");
179     strcat(temp, middleName);
180     strcat(temp, " ");
181     strcat(temp, lastName);
182
183     /* Split the temporary string into words */
184     token = strtok(temp, " ");
185     while (token != NULL && count < 3)
186     {
187         words[count++] = token;
188         token = strtok(NULL, " ");
189     }
190
191     /* Join the words from last to first */
192     strcpy(reverseName, "");
193     for (i = count - 1; i >= 0; i--)
194     {
195         strcat(reverseName, words[i]);
196         if (i > 0)
197             strcat(reverseName, " ");
198     }
199 }
200
201 /* Task 6: count vowels, consonants, digits and spaces */
202 void countCharacters(const char fullName[], int *vowels, int *consonants,
203                    int *digits, int *spaces)
204 {
205     size_t i, len = strlen(fullName);
206     char ch;
207
208     *vowels = *consonants = *digits = *spaces = 0;
209
210     for (i = 0; i < len; i++)
211     {

```

```

212     ch = tolower((unsigned char)fullName[i]);
213
214     if (isalpha((unsigned char)ch))
215     {
216         if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u')
217             (*vowels)++;
218         else
219             (*consonants)++;
220     }
221     else if (isdigit((unsigned char)ch))
222         (*digits)++;
223     else if (isspace((unsigned char)ch))
224         (*spaces)++;
225 }
226 }
227
228 /* Task 7: case-insensitive search; returns index of match, or -1 */
229 void searchWord(const char fullName[], const char word[], int *searchResult)
230 {
231     char nameLower[FULL_SIZE], wordLower[NAME_SIZE];
232     char *found;
233     size_t i;
234
235     for (i = 0; fullName[i] != '\0'; i++)
236         nameLower[i] = tolower((unsigned char)fullName[i]);
237     nameLower[i] = '\0';
238
239     for (i = 0; word[i] != '\0'; i++)
240         wordLower[i] = tolower((unsigned char)word[i]);
241     wordLower[i] = '\0';
242
243     found = strstr(nameLower, wordLower);
244     if (found != NULL)
245         *searchResult = (int)(found - nameLower);
246     else
247         *searchResult = -1;
248 }
249
250 /* Display the menu */
251 void displayMenu(void)
252 {
253     printf("\n----- MENU ----- \n");
254     printf("1. Display full name\n");
255     printf("2. Find length of full name\n");
256     printf("3. Copy full name to another string\n");
257     printf("4. Compare first name and last name\n");
258     printf("5. Display name in reverse order\n");
259     printf("6. Count vowels, consonants, digits, spaces\n");
260     printf("7. Search a word in full name\n");
261     printf("8. Exit\n");
262     printf("----- \n");
263 }
264
265 /* Discard leftover characters in the input buffer */
266 void clearInputBuffer(void)
267 {
268     int c;
269     while ((c = getchar()) != '\n' && c != EOF)
270         ;
271 }

```

9. Sample Output

Sample run with First Name = Rudra, Middle Name = Pratap and Last Name = Singh. (The menu is displayed again before every choice; it is shown only once below to keep the output short.)

```
=====
      STUDENT NAME OPERATIONS
=====
Enter first name   : Rudra
Enter middle name  : Pratap
Enter last name    : Singh

----- MENU -----
1. Display full name
2. Find length of full name
3. Copy full name to another string
4. Compare first name and last name
5. Display name in reverse order
6. Count vowels, consonants, digits, spaces
7. Search a word in full name
8. Exit
-----
Enter your choice (1-8): 1
Full name           : Rudra Pratap Singh

Enter your choice (1-8): 2
Length of full name (with spaces) : 18

Enter your choice (1-8): 3
Copied string       : Rudra Pratap Singh

Enter your choice (1-8): 4
First name and last name are DIFFERENT.

Enter your choice (1-8): 5
Reverse order       : Singh Pratap Rudra

Enter your choice (1-8): 6
Vowels             : 5
Consonants         : 11
Digits             : 0
Spaces             : 2

Enter your choice (1-8): 7
Enter the word to search: pratap
The word "pratap" FOUND in the full name (position 7).

Enter your choice (1-8): 7
Enter the word to search: xyz
The word "xyz" NOT FOUND in the full name.

Enter your choice (1-8): 9
Invalid choice! Please enter a number between 1 and 8.

Enter your choice (1-8): 8
Exiting the program. Thank you!
```

10. Result Verification

Operation	Expected	Program Output
Full name	Rudra Pratap Singh	Rudra Pratap Singh
Length (with spaces)	$5 + 1 + 6 + 1 + 5 = 18$	18
Reverse order	Singh Pratap Rudra	Singh Pratap Rudra
Vowels	u, a, a, a, i = 5	5
Consonants	16 letters – 5 vowels = 11	11
Spaces	2	2
First vs Last name	Different	DIFFERENT

11. Conclusion

The program successfully performs all the required operations on a student's name. Each task is implemented as a separate user-defined function, and pointers are used wherever a function modifies a string or returns results. The project helped in understanding string handling functions of <string.h>, character functions of <ctype.h>, modular programming and call by reference in C.